

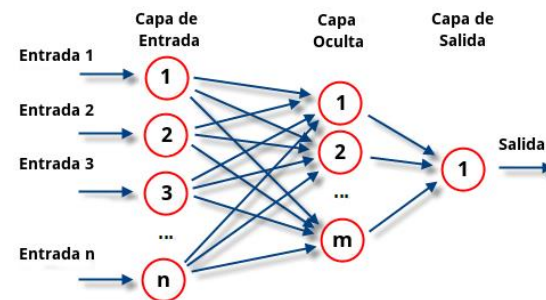
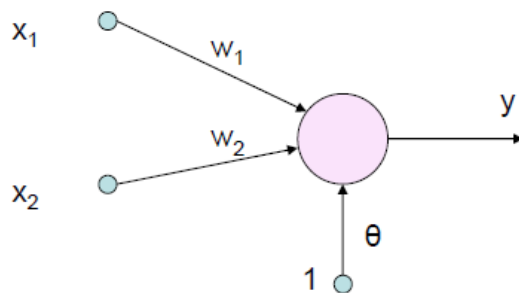


PERCEPTRÓN Y ARQUITECTURAS MULTICAPAS

Abraham Sánchez López
FCC/BUAP
Grupo MOVIS

Introducción

- Un perceptrón es una neurona única que clasifica un conjunto de entradas en una de dos categorías (generalmente 1 o -1).
- Si las entradas tienen forma de cuadrícula, se puede usar un perceptrón para reconocer imágenes visuales de formas.
- El perceptrón usualmente usa una función de paso, que devuelve 1 si la suma ponderada de las entradas excede un umbral, y 0 en caso contrario.
- Cuando las capas del perceptrón se combinan entre sí, forman una arquitectura multicapa, y esto proporciona la complejidad requerida del procesamiento de la red neuronal.
- Los perceptrones multicapa (Multi-Layer Perceptrons, MLP) son la arquitectura más utilizada para redes neuronales.

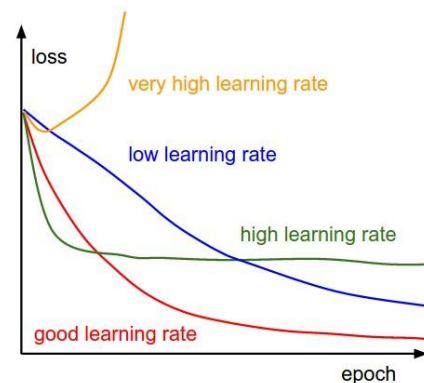


Propagación hacia adelante y hacia atrás, I

- El procesamiento desde la capa de entrada a las capas ocultas y luego a la capa de salida se denomina propagación hacia adelante (forward propagation).
- La suma(entrada*pesos) + sesgo se aplica en cada capa y luego el valor de la función de activación se propaga a la siguiente capa. La siguiente capa puede ser otra capa oculta o la capa de salida.
- La construcción de redes neuronales utiliza una gran cantidad de capas ocultas para dar lugar a la red neuronal profunda (Deep Neural Network, DNN).
- Una vez que se llega a la salida, en la última capa (la capa de salida), calculamos el error (la salida predicha menos la salida original).
- Este error es necesario para corregir los pesos y sesgos utilizados en la propagación directa.
- Aquí es donde se usa la función derivada. La cantidad de peso que debe cambiarse está determinada por el descenso del gradiente.
- El proceso de retropropagación (backpropagation) utiliza la derivada parcial de la función de activación de cada neurona para identificar la pendiente (o gradiente) en la dirección de cada uno de los pesos entrantes.

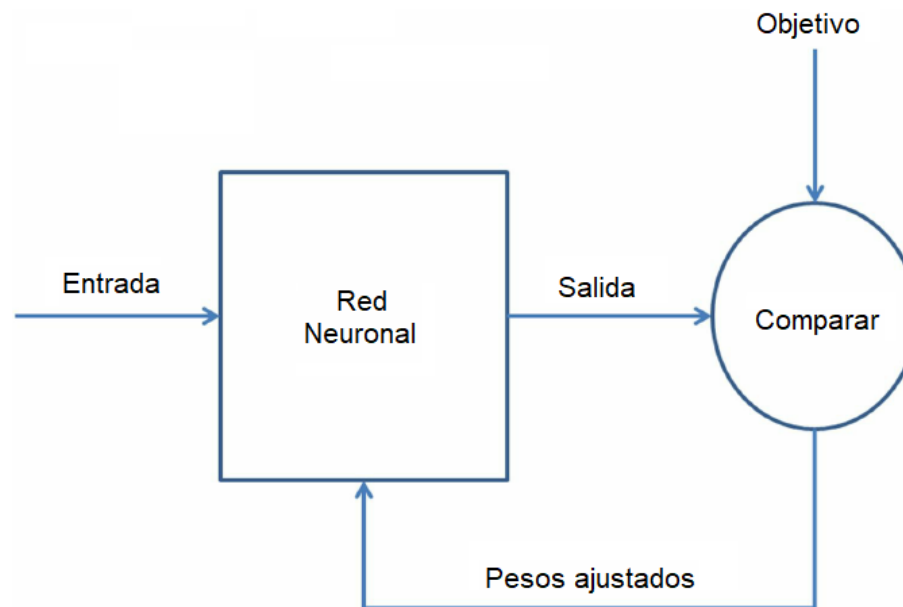
Propagación hacia adelante y hacia atrás, II

- El gradiente sugiere cuán abruptamente se reducirá o aumentará el error para un cambio en el peso.
- La propagación hacia atrás sigue cambiando los pesos hasta que haya una mayor reducción en los errores en una cantidad conocida como la *tasa de aprendizaje*.
- La tasa de aprendizaje es un parámetro escalar, análogo al tamaño del paso en la integración numérica, que se usa para establecer la tasa de ajustes para reducir los errores más rápido.
- La tasa de aprendizaje se utiliza en la propagación hacia atrás durante el ajuste de pesos y sesgos.
- A mayor velocidad de aprendizaje, más rápido el algoritmo reducirá los errores y más rápido será el proceso de entrenamiento:



Una red neuronal paso a paso, I

- Tomaremos un enfoque paso a paso para comprender el paso hacia adelante y hacia atrás con una sola capa oculta.
- La capa de entrada tiene una neurona y la salida resolverá un problema de clasificación binaria (predice 0 o 1).
- En la siguiente figura se muestra un paso hacia adelante y hacia atrás con una sola capa oculta:



Una red neuronal paso a paso, II

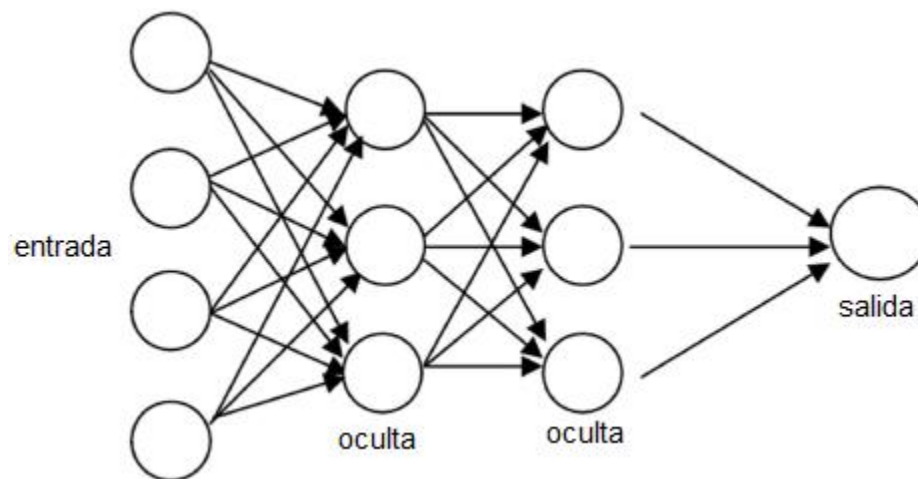
- A continuación, analicemos en detalle, paso a paso, todas las operaciones que se realizarán para el entrenamiento de la red:
 1. Tomar la entrada como una matriz.
 2. Inicializar los pesos y sesgos con valores aleatorios. Esta es una vez y seguiremos actualizándolas con el proceso de propagación de errores.
 3. Repetir los pasos del 4 al 9 para cada patrón de entrenamiento (presentado en orden aleatorio), hasta que se minimice el error.
 4. Aplicar las entradas a la red.
 5. Calcular la salida para cada neurona desde la capa de entrada, a través de las capas ocultas, hasta la capa de salida.
 6. Calcular el error en las salidas: real menos predicho.
 7. Usar el error de salida para calcular las señales de error para las capas anteriores. La derivada parcial de la función de activación se usa para calcular las señales de error.
 8. Usar las señales de error para calcular los ajustes de peso.
 9. Aplicar los ajustes de peso.

Una red neuronal paso a paso, III

- Los pasos 4 y 5 son propagación hacia adelante y los pasos 6 a 9 son propagación hacia atrás.
- La tasa de aprendizaje es la cantidad con la que se actualizan los pesos, se controla mediante un parámetro de configuración.
- El paso completo de ida y vuelta se llama ciclo de entrenamiento o época.
- Los pesos y sesgos actualizados se utilizan en el próximo ciclo.
- Seguimos entrenando recursivamente hasta que el error sea muy mínimo.
- Cubriremos más sobre la propagación hacia adelante y hacia atrás en detalle a lo largo de esta parte del curso.

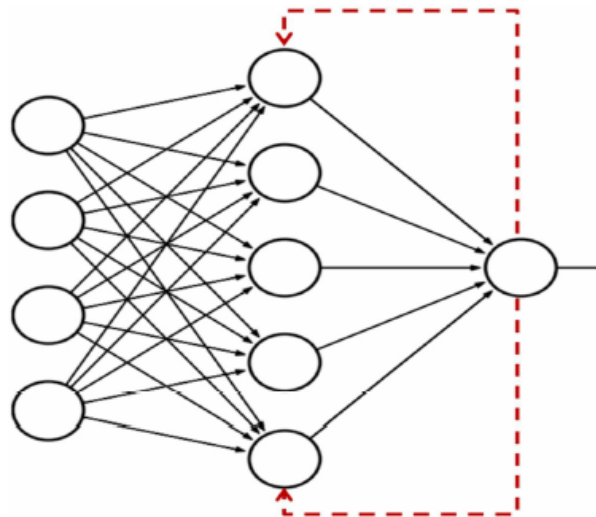
Redes feed-forward y feedback, I

- El flujo de las señales en las redes neuronales puede ser en una sola dirección o en recurrencia.
- En el primer caso, llamamos a la arquitectura de red neuronal feed-forward, ya que las señales de entrada se alimentan a la capa de entrada, luego, después de ser procesadas, se reenvían a la siguiente capa, tal como se muestra en la siguiente figura.
- Las MLP y las funciones de base radial también son buenos ejemplos de redes feed-forward. En la siguiente figura se muestra una arquitectura MLP:



Redes feed-forward y feedback, II

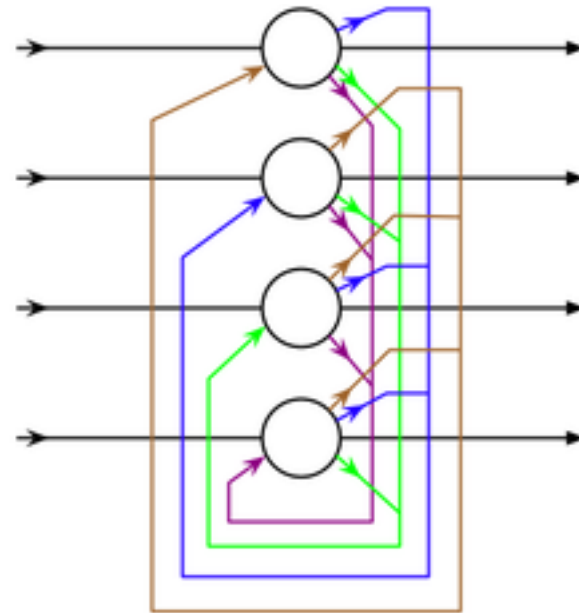
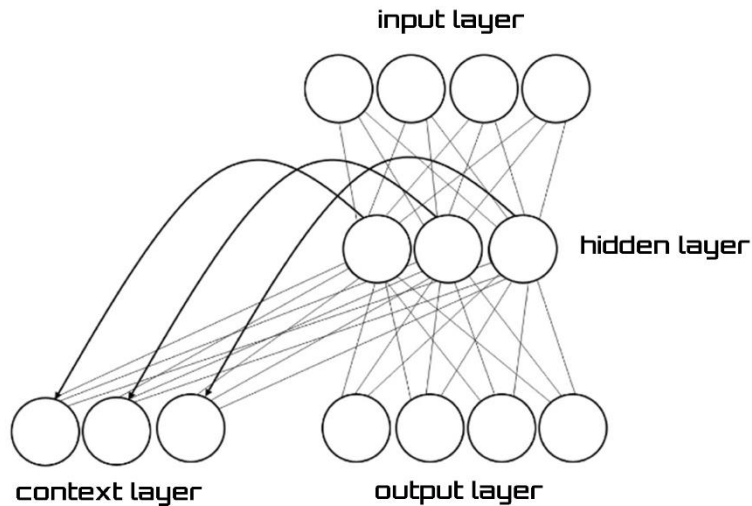
- Cuando la red neuronal tiene algún tipo de recurrencia interna, lo que significa que las señales se retroalimentan a una neurona o capa que ya ha recibido y procesado esa señal, la red es del tipo feedback, como se muestra en la siguiente imagen:



- La razón especial para agregar recurrencia en una red es la producción de un comportamiento dinámico, particularmente cuando la red aborda problemas que involucran series de tiempo o reconocimiento de patrones, que requieren una memoria interna para reforzar el proceso de aprendizaje.

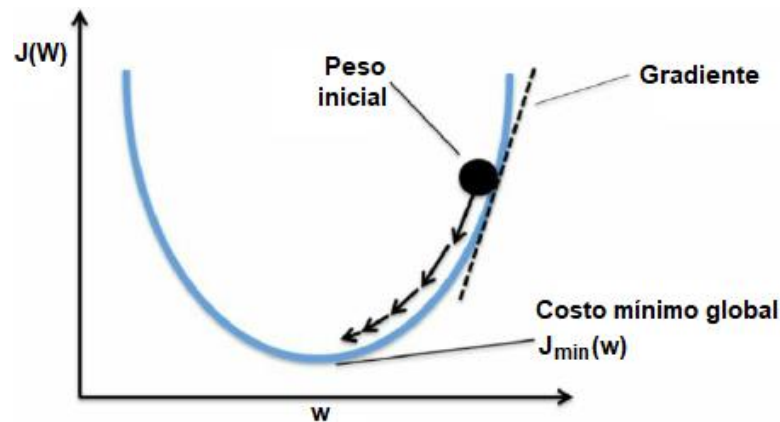
Redes feed-forward y feedback, III

- Sin embargo, tales redes son particularmente difíciles de entrenar, y finalmente no pueden aprender.
- La mayoría de las redes de retroalimentación son de una sola capa, como las redes Elman y Hopfield, pero es posible construir una red recurrente multicapa, como las redes echo y recurrentes MLP.



Gradiente descendiente, I

- El gradiente descendiente es un enfoque iterativo para la corrección de errores en cualquier modelo de aprendizaje.
- Para redes neuronales durante la retropropagación, el proceso de iterar la actualización de pesos y sesgos con los tiempos de error derivados de la función de activación es el enfoque de gradiente descendiente.
- El tamaño del paso de descenso más empinado se reemplaza por un tamaño similar al del paso anterior.
- El gradiente se define básicamente como la pendiente de la curva y es la derivada de la función de activación:



Gradiente descendiente, II

- El objetivo de derivar el gradiente descendiente en cada paso es encontrar el costo mínimo global, donde el error es el más bajo.
- Y aquí es donde el modelo se adapta bien a los datos y las predicciones son más precisas.
- El gradiente descendiente se puede realizar tanto para el lote completo como para el estocástico.
- En el gradiente descendiente de lote completo, el gradiente se calcula para el conjunto de datos de entrenamiento completo, mientras que el Gradiente Descendiente Estocástico (Stochastic Gradient Descent, SGD) toma una sola muestra y realiza el cálculo del gradiente.
- También puede tomar mini lotes y realizar los cálculos. Una ventaja del SGD es el cálculo más rápido de gradientes.

Taxonomía de las redes neuronales, I

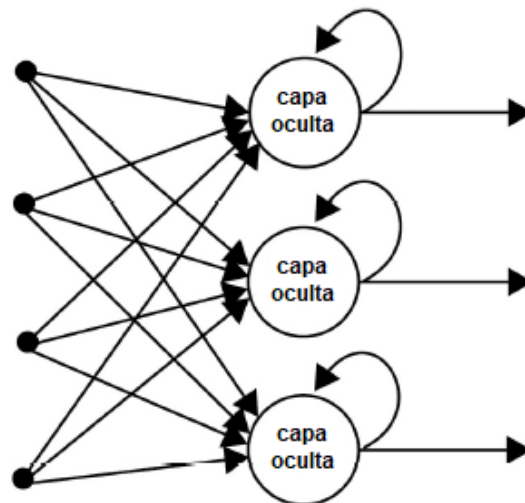
- La base básica de las RNA es la misma, pero se han diseñado varios modelos de redes neuronales durante su evolución. Los siguientes son algunos de los modelos de RNA:
- El elemento lineal adaptativo (ADALINE) es un perceptrón simple que solo puede resolver problemas lineales. Cada neurona toma la suma lineal ponderada de las entradas y la pasa a una función bipolar, que produce un $+1$ o -1 dependiendo de la suma. La función verifica la suma de las entradas pasadas y si la red es ≥ 0 , es $+1$, de lo contrario es -1 .
- Multiple ADALINEs (MADALINE), es una red multicapa de unidades ADALINE.
- Los perceptrones son redes neuronales de capa única (neurona única o unidad), donde la entrada es multidimensional (vector) y la salida es una función de la suma de peso de las entradas.
- La red de función de base radial es una RNA donde se utiliza una función de base radial como función de activación. La salida de red es una combinación lineal de funciones de base radial de las entradas y algunos parámetros neuronales.

Taxonomía de las redes neuronales, II

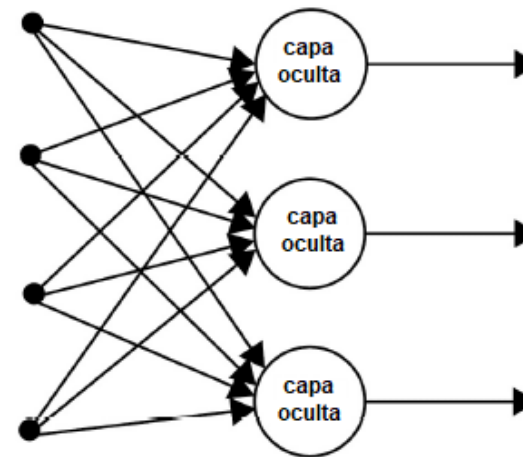
- Las feed-forward es la forma más simple de redes neuronales. Los datos que se procesan en capas sin ningún ciclo son ciclos.
- Se estudian a menudo las siguientes redes feedforward en un curso básico e intermedio:
 - Autoencoder
 - Probabilístico
 - Tiempo de retardo
 - Convolutacional
- Las redes neuronales recurrentes (RNN), a diferencia de las redes de alimentación directa, propagan datos hacia adelante y hacia atrás desde etapas de procesamiento posteriores a etapas anteriores.
- Los siguientes son los tipos de RNN más conocidos:
 - Redes Hopfield
 - Máquina Boltzmann

Taxonomía de las redes neuronales, III

- Mapas autoorganizados (Self Organizing Maps, SOM)
- Memoria asociativa bidireccional (Bidirectional Associative Memory, BAM)
- Memoria a corto y largo plazo (Long Short Term Memory, LSTM)
- Las siguientes imágenes muestran (a) Red neuronal recurrente y (b) Red neuronal de propagación (forward):



(a) Red neuronal recurrente



(b) Red neuronal de propagación

Ejemplo simple con R, I

- Considera un conjunto de datos simple de un cuadrado de números, que se usará para entrenar una función de red neuronal en R y luego probar la precisión de la red neuronal construida:

Entrada	Salida
0	0
1	1
2	4
3	9
4	16

Entrada	Salida
5	25
6	36
7	49
8	64
9	81
10	100

- Nuestro objetivo es establecer los pesos y el sesgo para que el modelo pueda hacer lo que se está haciendo aquí. La salida debe modelarse en función de una entrada y la función puede usarse en el futuro para determinar la salida en función de una entrada:

Ejemplo simple con R, II

```
# Elige las bibliotecas que se usarán
install.packages("neuralnet")
library(neuralnet)
# Establece el directorio de trabajo para los datos de entrenamiento
setwd("C:/R")
getwd()
# Lee el archivo de entrada
mydata=read.csv('Squares.csv',sep="," ,header=TRUE)
mydata
attach(mydata)
names(mydata)
# Entrena el modelo basado en la salida de entrada
model=neuralnet(formula = Output~Input,
data = mydata,
hidden=10,
threshold=0.01 )
print(model)
# Permite trazar y ver las capas
plot(model)
#Comprueba los datos: reales y previstos
final_output=cbind (Input, Output,
as.data.frame(model$net.result) )
colnames(final_output) = c("Input", "Expected Output",
"Neural Net Output" )
print(final_output)
```

Análisis del código, I

- Para comprender todos los pasos del código que acabamos de proponer, los analizaremos en detalle.
- No te preocupes si algunos pasos parecen poco claros en este momento, podrás verlo en los siguientes ejemplos. Primero, se mostrará el fragmento de código, y la explicación seguirá:

```
library("neuralnet")
```

- La línea en R incluye la biblioteca `neuralnet()` en nuestro programa. `neuralnet()` es parte de la Red Integral de Archivo R (CRAN), que contiene numerosas bibliotecas R para diversas aplicaciones.

```
mydata=read.csv('Squares.csv',sep="," ,header=TRUE)
```

```
mydata
```

```
attach(mydata)
```

```
names(mydata)
```

- Esto lee el archivo CSV con separador (coma) y el encabezado es la primera línea del archivo. `names()` mostraría el encabezado del archivo.

Análisis del código, II

```
model=neuralnet(formula = Output~Input,  
data = mydata,  
hidden=10,  
threshold=0.01 )
```

- El entrenamiento de la salida con respecto a la entrada ocurre aquí. La biblioteca `neuralnet()` pasa los nombres de las columnas de entrada y salida (`output ~ input`), el conjunto de datos que se utilizará, el número de neuronas en la capa oculta y los criterios de detención (`umbral, threshold`).
- En la siguiente dirección puedes consultar más detalles del paquete `neuralnet`:

<https://cran.r-project.org/web/packages/neuralnet/neuralnet.pdf>

- Es interesante checar la documentación de `neuralnet`:

```
neuralnet(formula, data, hidden = 1, threshold = 0.01, stepmax = 1e+05, rep = 1,  
startweights = NULL, learningrate.limit = NULL, learningrate.factor = list(minus = 0.5,  
plus = 1.2), learningrate=NULL, lifesign = "none", lifesign.step = 1000, algorithm =  
"rprop+", err.fct = "sse", act.fct = "logistic", linear.output = TRUE, exclude = NULL,  
constant.weights = NULL, likelihood = FALSE)
```

Análisis del código, III

- Después de dar un breve vistazo a la documentación del paquete, revisemos las líneas restantes del ejemplo de código propuesto:

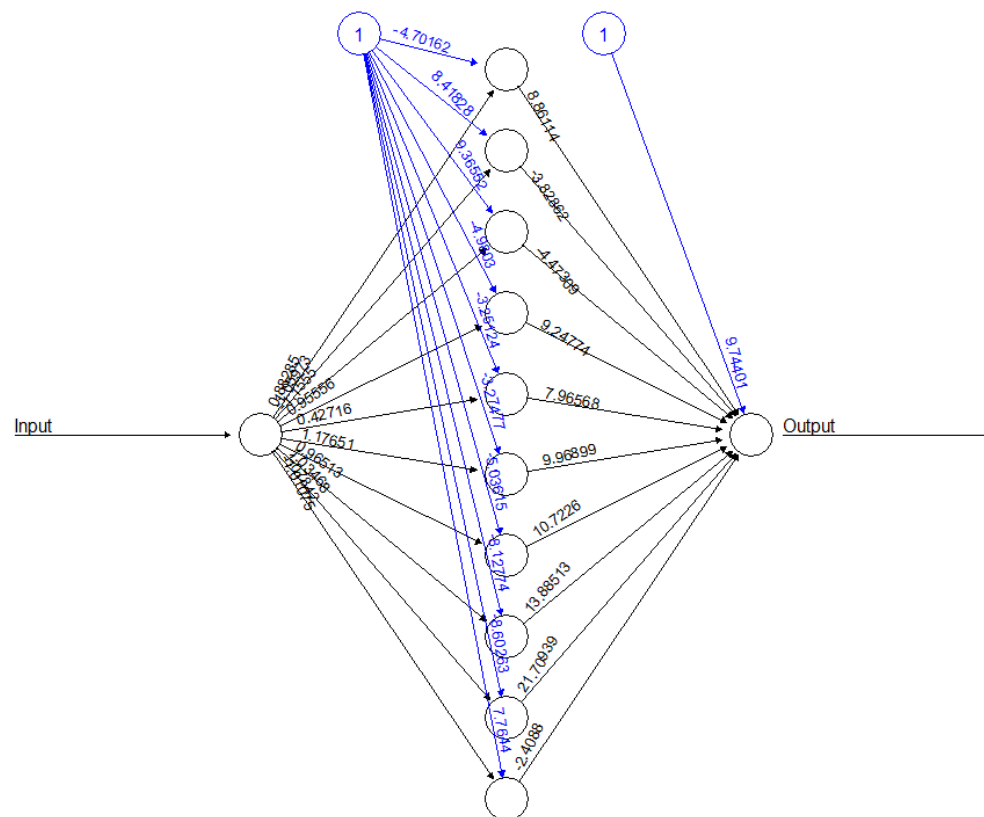
`print(modelo)`

- Este comando imprime el modelo que se acaba de generar, de la siguiente manera:

```
$result.matrix
              [,1]
error          1.465823e-02
reached.threshold 9.741172e-03
steps          6.602000e+03
Intercept.to.llayhid1 -4.701616e+00
Input.to.llayhid1      8.828468e-01
Intercept.to.llayhid2  8.418285e+00
Input.to.llayhid2     -1.059735e+00
Intercept.to.llayhid3  9.365515e+00
Input.to.llayhid3     -1.155504e+00
Intercept.to.llayhid4 -4.980304e+00
Input.to.llayhid4      9.555590e-01
Intercept.to.llayhid5 -3.251240e+00
Input.to.llayhid5      4.271633e-01
Intercept.to.llayhid6 -3.274771e+00
Input.to.llayhid6      1.176509e+00
Intercept.to.llayhid7 -5.036150e+00
Input.to.llayhid7      9.651267e-01
Intercept.to.llayhid8 -8.127741e+00
Input.to.llayhid8      1.034677e+00
Intercept.to.llayhid9 -8.602629e+00
Input.to.llayhid9      1.078425e+00
Intercept.to.llayhid10 7.764399e+00
Input.to.llayhid10    -1.010749e+00
Intercept.to.Output    9.744008e+00
llayhid1.to.Output     8.861140e+00
llayhid2.to.Output    -3.828616e+00
llayhid3.to.Output    -4.473089e+00
llayhid4.to.Output     9.247741e+00
llayhid5.to.Output     7.965684e+00
llayhid6.to.Output     9.968992e+00
llayhid7.to.Output     1.072260e+01
llayhid8.to.Output     1.388513e+01
llayhid9.to.Output     2.170939e+01
llayhid10.to.Output    -2.408796e+00
```

Análisis del código, IV

- Volvamos al análisis de código:
`plot(model)`
- Este comando anterior grafica (plotea) la red neuronal para nosotros, de la siguiente manera:



Análisis del código, V

```
final_output=cbind (Input, Output,  
as.data.frame(model$net.result) )  
colnames(final_output) = c("Input", "Expected Output",  
"Neural Net Output" )  
print(final_output)
```

- Este código anterior imprime la salida final, comparando la salida predicha y la real como:

```
> print(final_output)
```

	Input	Expected Output	Neural Net Output
1	0	0	-0.08375051
2	1	1	1.13141162
3	2	4	3.93730999
4	3	9	9.00228157
5	4	16	16.01980747
6	5	25	25.00124795
7	6	36	35.98077017
8	7	49	49.01704095
9	8	64	63.99336727

Ejemplo con la librería `nnet()`, I

- Para mejorar nuestra práctica con la biblioteca `nnet`, analicemos otro ejemplo.
- Esta vez utilizaremos los datos recopilados en un restaurante a través de entrevistas con clientes.
- Se pidió a los clientes que dieran una puntuación a los siguientes aspectos: servicio, ambiente y comida. También se les preguntó si dejarían la propina sobre la base de estos puntajes.
- En este caso, el número de entradas es 2 y la salida es un valor categórico (Tip = 1 y No-tip = 0).
- El archivo de entrada a utilizar se muestra en la siguiente tabla (resumido!):

No	CustomerWillTip	Service	Ambience	Food	TipOrNo
1	1	4	4	5	Tip
2	1	6	4	4	Tip
:	:	:	:	:	:
30	0	1	3	2	No-tip

Ejemplo con la librería nnet(), II

- Este es un problema de clasificación con tres entradas y una salida categórica. Abordaremos el problema con el siguiente código:
- Primero hay que instalar las librerías: NeuralNetTools y nnet.

Elige las bibliotecas que se usarán

```
library(NeuralNetTools)
```

```
library(nnet)
```

Establece el directorio de trabajo para los datos de entrenamiento

```
setwd("C:/R")
```

```
getwd()
```

Lee el archivo de entrada

```
mydata=read.csv('RestaurantTips.csv',sep="," ,header=TRUE)
```

```
mydata
```

```
attach(mydata)
```

```
names(mydata)
```

Entrena el modelo basado en la salida de entrada

```
model=nnet(CustomerWillTip~Service+Ambience+Food,
```

```
data = mydata,
```

```
size=5,
```

```
rang=0.1,
```

```
decay=5e-2,
```

```
maxit=5000)
```

Ejemplo con la librería nnet(), III

```
print(model)
plotnet(model)
garson(model)
```

- Para comprender todos los pasos del código que acabamos de proponer, los analizaremos en detalle. Primero, se mostrará el fragmento de código y seguirá la explicación.

```
library(NeuralNetTools)
library(nnet)
```

- Esto incluye las bibliotecas NeuralNetTools y nnet() para nuestro programa.

```
# Establece el directorio de trabajo para los datos de entrenamiento
setwd("C:/R")
getwd()
# Lee el archivo de entrada
mydata=read.csv('RestaurantTips.csv',sep=";",header=TRUE)
mydata
attach(mydata)
names(mydata)
```

Ejemplo con la librería nnet(), IV

- Esto establece el directorio de trabajo y lee el archivo CSV de entrada.

```
# Entrena el modelo basado en la salida de entrada
model=nnet(CustomerWillTip~Service+Ambience+Food,
data = mydata,
size=5,
rang=0.1,
decay=5e-2,
maxit=5000)
print(model)
```

- Esto llama a la función nnet() con los argumentos pasados. La salida es la siguiente.
nnet() **procesa la propagación hacia adelante y hacia atrás hasta la convergencia:**

```
# weights: 26
initial value 7.513817
iter 10 value 6.019039
iter 20 value 5.385541
iter 30 value 5.288950
iter 40 value 5.231530
iter 50 value 5.220029
iter 60 value 5.217473
iter 70 value 5.216748
iter 80 value 5.216665
final value 5.216665
converged
```

- Para una descripción más detallada de nnet, revisar en la web.

Ejemplo con la librería `nnet()`, V

- Después de dar un breve vistazo a la documentación del paquete, revisemos las líneas restantes de la propuesta en el siguiente ejemplo de código:

```
print(model)
```

- Este comando imprime los detalles de `net()` de la siguiente manera:

```
> print(model)
a 3-5-1 network with 26 weights
inputs: Service Ambience Food
output(s): CustomerWillTip
options were - decay=0.05
```

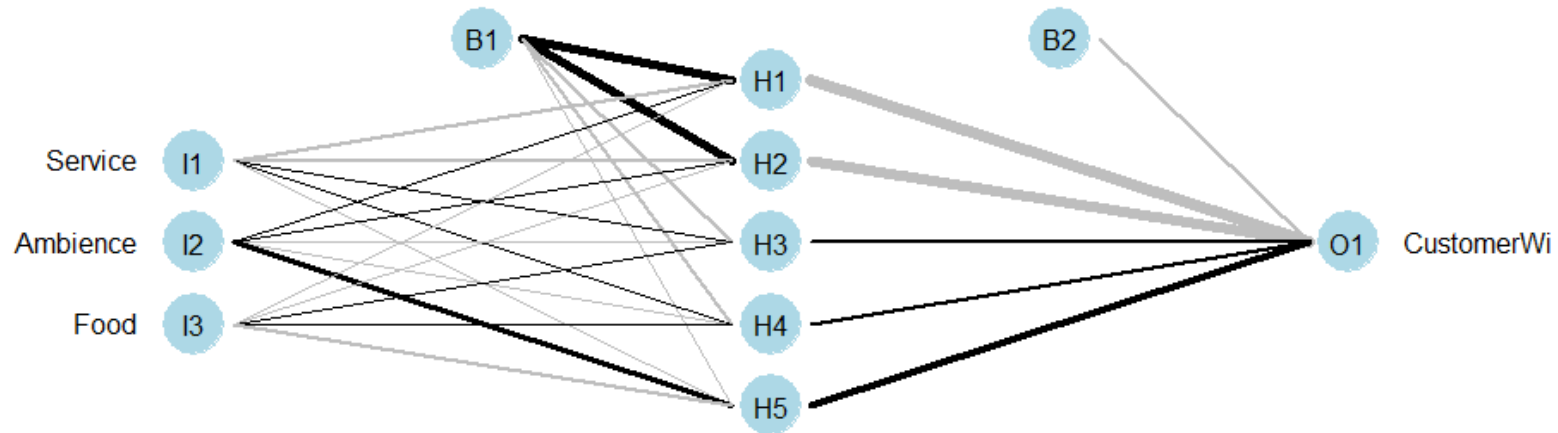
- Para plotear (graficar) el modelo, usa el siguiente comando:

```
plotnet(model)
```

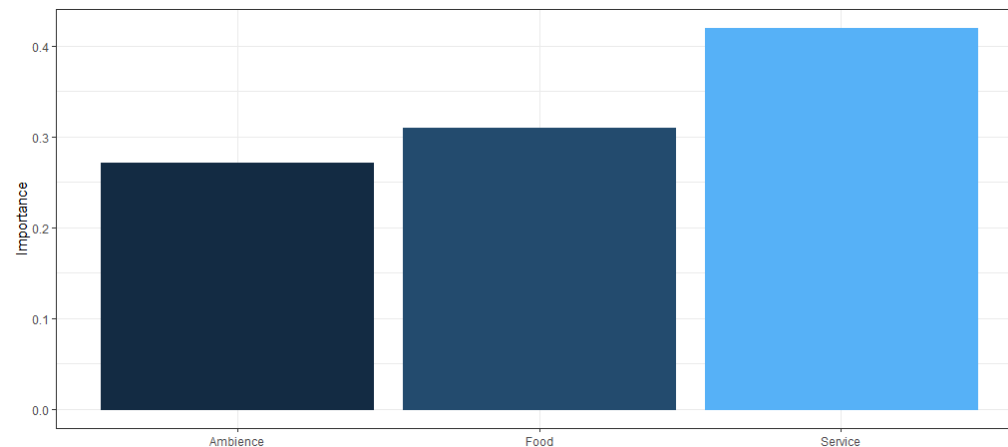
- La grafica del modelo se muestra en el acetato 28; hay cinco nodos en la única capa oculta.
- Usando `NeuralNetTools`, es posible obtener la importancia relativa de las variables de entrada en redes neuronales usando el algoritmo garson:

```
garson(model)
```

Ejemplo con la librería nnet(), VI



- Este comando imprime los diversos parámetros de entrada y su importancia para la predicción de salida, como se muestra en la siguiente figura:



Ejemplo con la librería `nnet()`, VII

- A partir de la tabla obtenida de la aplicación del algoritmo Garson, es posible notar que, en la decisión de dar la propina, el servicio recibido por los clientes tiene la mayor influencia.
- Hemos visto dos bibliotecas de redes neuronales en R y las hemos usado en ejemplos simples.
- Profundizaríamos con varios casos de uso práctico a lo largo de esta parte del curso de Técnicas de Inteligencia Artificial.